Higher-Order Functions & Environment Diagrams Meta

Computer Science Mentors 61A

September 7, 2026 – September 11, 2026

Example Timeline

- Intros, Ice Breakers, Expectations, Logistics [5 min]

  Don't be afraid to spend some time on this! It'll likely make your job in the future easier! Prospective mentees will also get to know how CSM works and doing icebreakers will make it a welcoming experience for all of them but try to keep them kind of short since this is not their official session.

  Also, something we want to do this semester more of is emphasize the conceptual topics and concepts in 61A like functional abstraction and more. So make sure to talk about them regularly throughout all of your teaching sessions and tell junior mentors to also emphasize them as well.

- Environment Diagrams Mini Lecture + Q1 [10 mins]

  1. You can use Q1 as an example for the mini-lecture

  2. A lot of students (even those with significant programming background) get confused by env. diagrams. It's probably worth doing the minilecture even if you have advanced students.

  3. You should address when we need to make a new frame in an environment diagram.

- Higher Order Functions Overview + Reasoning + Q1 + Q2 [15 mins]

- Problems (You pick which ones): HOF

  1. Foo: has an example of a function which is returned along with return in the middle of function and returning None

  2. Compose: easy example to build skill in writing lambda functions.

  3. WholeSum: Another HOF example and digit manipulation and while loop practice

  4. Alternator: HOF example with while loop practice

  5. Curryforever: An example with writing code for a function which takes in a variable number of arguments; the number of arguments that will be taken is input earlier

Created by Cadence Loh and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

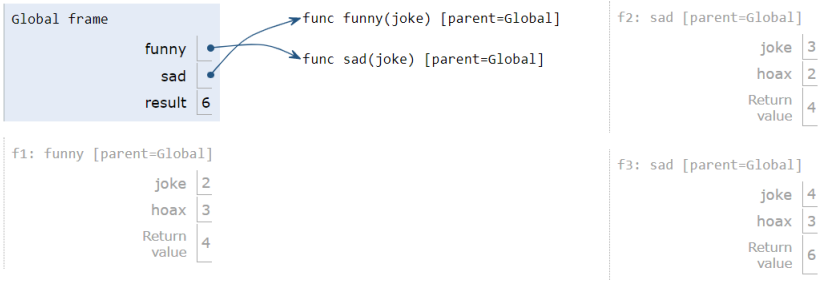
1 Environment Diagrams

1. Draw the environment diagram that results from running the following code.

```
def funny(joke):
    hoax = joke + 1
    return funny(hoax)

def sad(joke):
    hoax = joke - 1
    return hoax + hoax

funny, sad = sad, funny
result = funny(sad(2))
```



https://tinyurl.com/y5lc4fez

Suggested Time: 10 min; Difficulty: Medium

- Make sure that the students understand how Python looks for a value of a variable, from local (to parent(s)) to global.
- Make sure your students understand the difference between an intrinsic name and a bound name

  Intrinsic: For user defined functions, this intrinsic name is the name used in the **`def`** statement

  Bound: Names of variables that point to the function object. A function can have many bound names, and the bound names of a function can often change.

2 Higher-Order Functions

1. What are higher-order functions? Why and where do we use lambda and higher-order functions? Can you give a practical example of where we would use a HOF?

   Higher-order functions are functions that does at least one of the following: take at least one or more functions as arguments and returns a function. In practice, we use lambda functions to pass code as data in a concise manner. Lambda functions can be passed as arguments to higher-order functions. Higher order functions serve as a tool of abstraction, allowing us to simplify repeated actions into one function that we can use over and over again. Students can have varying answers for practice uses of HOFs, though here are some suggestions for the average student coming across this worksheet:

   - Our method signature is composed of one parameter, but we wish to use a higher order function with more parameters to abstract extra steps.
   - When our function is long and complex; easier to read code when it's organized into several different higher order functions.

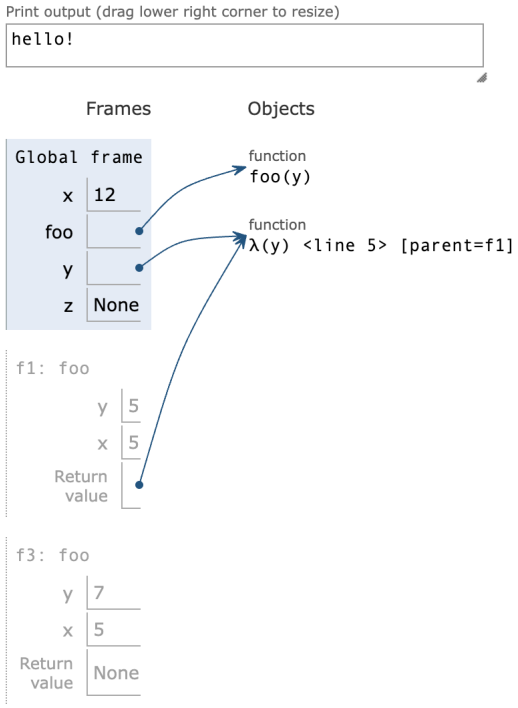
   Suggested Time: 5 mins

   Treat this as more of a guiding question to start discussion on HOFs, like talk about it and do not treat it like other CSM questions. Try to give examples when explaining HOFs. Some examples are linked at this link: `https://tinyurl.com/5fsd3h8b`. One specific example to illustrate the use of lambdas is the optional `key` parameter for **`min`** and **`max`** functions, but just as a side note you could also pass in a function defined using the keyword def for the `key` parameter. Might want to show examples that illustrate each of the bullet points in action.

2. Give the environment diagram and console output that result from running the following code.

```
x = 20
def foo(y):
    x = 5
    if y == 5:
        return lambda y: x + y
    else:
        print('hello!')

y = foo(5)
x = y(7)
z = foo(7)
```



https://tinyurl.com/4dkbpnyc

Suggested Time: 7 mins; Difficulty: Easy

- Emphasize that if line with return statement is reached, then expression after return keyword will be evaluated and that value is then returned to the environment/frame in which the function returning the value was called in.
- Also, emphasize that if no return statement is encountered while calling a function with certain arguments then the return value is None.

3. Draw the environment diagram that results from running the code.

```
def a(y):
    d = 1
    b = lambda x: y(x)
    e = lambda x: x(3)
    return e(b)
d = 5
a(lambda x: 4 - x + d)
```

https://goo.gl/9vxEwv

Suggested Time: 10 min; Difficulty: Medium

- Make sure that students understand how lambda functions are expressed in the environment diagram.
- Make sure to stress variables in different scopes.

4. Implement `compose`, a function which takes in two functions `f` and `g`, both of which take in one argument each. `compose` returns a function which can take one argument as well. When this returned function is called with an argument, `f(g(x))` is returned.

```
def compose(f, g):
    """
    >>> a = compose(lambda x: x * x, lambda x: x + 4)
    >>> a(2)
    36
    """

    _______ lambda _________________

    return lambda x: f(g(x))
```

Suggested Time: 5 mins; Difficulty: Easy

5. Write a function, `whole_sum`, which takes in an integer, `n`. It returns another function which takes in an integer, and returns `True` if the digits of that integer sum to `n` and `False` otherwise.

```
def whole_sum(n):
    """
    >>> whole_sum(21)(777)
    True
    >>> whole_sum(142)(10010101010)
    False
    """
    def check(x):

        ____________________

        while _______________:

            last = ____________________

            ____________________________

            ____________________________

        return ____________________

    return ____________________
```

```
def whole_sum(n):
    def check(x):
        total = 0
        while x > 0:
            last = x % 10
            x = x // 10
            total += last
        return total == n
    return check
```

Suggested Time: 8 Mins; Difficulty: Medium

- Remind your students that for HOFs, you must **`return`** the inner function (ie we must **`return`** `check` to use it).
- Also depending on the skill level of students in your section, a recap of digit manipulation may be needed (ie x // 10, x % 10, etc.)

6. Implement `make_alternator` which takes in two functions `f` and `g` and outputs a function. The returned function takes in a number `x` and the function goes through the numbers in the sequence {1, 2, 3, ... n} in ascending order; for each number in the sequence the function applies `f` if the number is odd and `g` if the number is even and then prints the result of applying `f` or `g` and moves on to the next number in the sequence.

```
def make_alternator(f, g):
    """
    >>> a = make_alternator(lambda x: x * x, lambda x: x + 4)
    >>> a(5)
    1
    6
    9
    8
    25
    """

    ___ alternator(x):

        ________________

        ________ i _______:

            if _______________:

                print(____)

            ___________________:

                _____________

            _____________

    ____________________
```

```
def alternator(x):
    i = 1
    while i <= x:
        if i % 2 == 1:
            print(f(i))
        else:
            print(g(i))
        i += 1
return alternator
```

Suggested Time: 8 mins; Difficulty: Medium

- Walk students through each iteration from 1 to x, and show how each of the two functions `f,g` alternate on incrementing inputs.
- Remember the general structure needed whenever a function must return a function.

7. Write a function, `curry_forever`, which takes in a two-argument function, `f`, and an integer, `arg_num`. It returns another function that helps in calling `f arg_num` number of times on input provided to this returned function.

```
def curry_forever(f, arg_num, base=0):
    """
    >>> g = curry_forever(lambda x, y: x + y, 4)
    >>> g(1)(2)(3)(4) # 1 + 2 + 3 + 4
    10
    """

    def helper(arg_num, amt):

        if arg_num == 0:

            ____________________________________________________

        return ____________________________________________

    ____________________________________________________________
```

```
def curry_forever(f, arg_num, base=0):
    def helper(arg_num, amt):
            if arg_num == 0:
                    return amt
            return lambda x: helper(arg_num - 1, f(amt, x))
    return helper(arg_num, base)
```

Suggested Time: 15 mins; Difficulty: Medium

- Students might be confused on how to build a function which takes in a variable number of arguments. If they are stuck, give them some hints on how to approach in general questions with skeleton code.